

[Diapositiva 0 – Introducción]

Buenas tardes.

Me presento: soy Álvaro Verdeguer, y hoy voy a exponer mi proyecto de fin de grado, titulado “Roig Arena”. Se trata de un sistema de gestión para una empresa, concretamente para un recinto multiusos, el Roig Arena. El objetivo es proporcionar una plataforma web que permita administrar eventos, venta de entradas y todo el flujo asociado a la reserva y compra de localidades también conocidos como asientos.

[Diapositiva 1 – Visión general del ecosistema]

Este TFG implementa un ecosistema completo para un recinto –en este caso, el Roig Arena–.

El sistema está diseñado para cubrir todas las necesidades de gestión de un espacio donde se celebran eventos, desde la publicación hasta la confirmación de asistencia.

[Diapositiva 1 (continuación) – Funcionalidades principales]

El ecosistema permite, entre otras cosas:

- Publicar eventos con toda su información (fecha, ubicación, plazas, artistas, etc.).
 - Reservar asientos para esos eventos, garantizando la integridad de los asientos.
 - Confirmar la posesión de las entradas mediante la generación de un código QR único por cada entrada.
 - Gestionar la expiración de reservas no pagadas y la liberación automática de asientos para impedir duplicados y errores.
-

[Diapositiva 5 – Requisitos del proyecto] (*Nota: no había texto previo*)

Para el correcto funcionamiento y despliegue del proyecto, se han establecido una serie de requisitos técnicos.

El sistema requiere un entorno compatible con contenedores Docker, acceso a terminal Bash, y puertos libres para servicios web y base de datos. Todos estos requisitos se automatizan mediante los scripts que se describen a continuación.

[Diapositiva 6 – Docker]

Docker es una tecnología de contenerización que nos permite empaquetar la aplicación y todas sus dependencias en un entorno aislado y portable.

Mediante los scripts `arena.sh` y `setup-arena.sh`, podemos desplegar el proyecto en

cualquier máquina que disponga de una terminal bash y Docker instalado, sin preocuparnos por diferencias en el sistema operativo o versiones de software.

[Diapositiva 7 – Sail]

Sail es una herramienta oficial de Laravel que facilita el desarrollo sobre contenedores Docker.

Actúa como una capa de abstracción que simplifica comandos como `./vendor/bin/sail up`, `sail artisan`, etc. Gracias a Sail, no es necesario conocer en profundidad Docker para levantar el entorno de desarrollo.

como podemos ver es una herramienta propietaria de laravel y cuenta con su propia documentación para guiarnos en su uso.

[Diapositiva 8 – Laravel: qué es y cómo organiza el funcionamiento de la página]

Laravel es la herramienta principal con la que he construido el backend, es decir, la parte invisible que hace funcionar la página web. Podemos imaginarlo como el "cerebro" del sistema: recibe las órdenes del usuario, procesa la información, habla con la base de datos y devuelve una respuesta.

Ahora vamos a hablar de como funciona laravel para no perdernos en detalles en las proximas diapositivas

1. **Las rutas** – existen las de API y las de Web , las primeras contienen una logica que devuelve un json y las otras cargan paginas enteras
2. **Los controladores** - se encargan de atender las peticiones de las rutas para conectar con la logica necesaria, si vas a eventos y entras al 2 pues te atendera el controlador de eventos entendiend que te refieres al 2, estan asociados.

Ejemplo concreto:

- Un usuario entra en `http://localhost/eventos/2`. La ruta está asociada al controlador `EventoController`.
- Ese controlador recibe la petición, entiende que el número `2` se refiere al evento con identificador 2, y va a la base de datos a buscar los datos de ese evento (nombre, fecha, asientos libres, etc.).
- Luego, el controlador elige una vista (una plantilla HTML) y le inyecta esos datos. El resultado es la página de detalle del evento que ve el usuario.

3. **Los modelos** – traducen consultas complejas a lo que yo califico como POO, osea en vez de hacer la consulta compleja y devolver datos, devuelves un objeto con los datos de la consulta de una manera mucho mas ordenada.

por ahora todos los backend suelen contar con esto.

[Diapositiva 9 – MySQL]

MySQL actúa como sistema gestor de base de datos relacional.

Almacena toda la información necesaria: usuarios, eventos, sectores, asientos, reservas, compras y entradas.

Elegi Mysql porque se integra muy bien con laravel y es rapido, a otra escala se tendria que migrar a otro sistema de base de datos.

[Diapositiva 10 – SETUP!] (*Introducción a los scripts de configuración*)

A continuación hablare del proceso de puesta en marcha del proyecto.

para esto contamos con dos scripts fundamentales que automatizan la configuración inicial y el reinicio del entorno.

[Diapositiva 11 – setup-arena.sh]

El script **setup-arena.sh** se encarga de levantar un contenedor Docker con todas las variables del sistema preconfiguradas.

De esta forma, conseguimos que no falte ningún requisito, ni de aplicaciones ni de cuentas de usuario. Todo viene predefinido para que no haya lugar a error durante la instalación. Es importante mencionar que se necesitan privilegios de administrador para instalar ciertos componentes.

[Diapositiva 12 – .env.example]

En Laravel, el archivo **.env** guarda la configuración del proyecto, como la base de datos o las claves de la aplicación.

El archivo `.env.example` es simplemente una plantilla que se incluye en el proyecto. Sirve para indicar qué variables necesita el proyecto, y ya viene en nuestro caso preconfigurada pero eso es porque es un lanzamiento de prueba.

Cuando otro desarrollador clona el proyecto, copia este archivo y crea su propio `.env` con sus valores.

Con esto evitamos almacenar contraseñas o información sensible y así mejorar la seguridad del proyecto.

[Diapositiva 13 – arena.sh]

El script `arena.sh` tiene una función diferente: **resetea el entorno por completo.**

- Vuelve a levantar los servicios Docker.
- Relanza MySQL y repuebla la base de datos con datos iniciales, partiendo de cero.
- También recrea la `APP_KEY` de Laravel.

Esta clave es un string aleatorio que usa el framework para cifrar cookies, sesiones y otros datos sensibles. Si falta, comandos y tests pueden fallar con una excepción `MissingAppKeyException`. Por eso `arena.sh` genera una nueva si no existe.

[Diapositiva 14 – Tests: introducción]

Ahora que tenemos establecida la base del proyecto, es momento de lanzar una batería de tests para asegurar que todo funciona correctamente. (aunque spoiler ya se ha hecho con los scripts anteriores)

El proyecto utiliza **PHPUnit**, una herramienta para hacer pruebas automáticas y comprobar que el código se comporta como se espera. Las dependencias y configuraciones se gestionan desde el archivo `composer.json`, que define las librerías y paquetes de PHP. Los tests se ejecutan con **Artisan**, la consola de comandos de Laravel, mediante `php artisan test`.

[Diapositiva 14.1 – Funcionamiento de un test]

Antes de continuar voy a explicar brevemente como es que funciona un test.

Un test simula una petición a una ruta de la API con unos datos concretos, como si estuviésemos haciendo una petición real, ya sea de un navegador o un cliente HTTP. Después, se comprueba la respuesta del sistema: en concreto, el código de estado HTTP,

para ver si ha resuelto 200 bien o con problemas

la estructura de los datos devueltos, para ver si es la correcta, y que la lógica de negocio se haya ejecutado correctamente. De esta forma se verifica que la funcionalidad devuelve datos y estados correctos o falsos, un tests puede aprobar fallando.

voy a hacer un recorrido de todos los tests muy brevemente, así cuando veamos las respuestas en el terminal, veremos que partes cumple nuestro sistema.

Diapositiva 15 – AuthTest

AuthTest comprueba el sistema de autenticación.

Se prueba el registro, el inicio de sesión correcto y el incorrecto, el cierre de sesión y el acceso al usuario con `/api/user`.

Se usa un usuario de prueba y se revisa que cada respuesta sea la esperada.

Diapositiva 16 – EventoTest

EventoTest verifica que la API devuelve bien la lista de eventos.

Se prueba con un usuario normal y con un administrador para ver que ambos reciben los eventos correctamente, aunque con posibles diferencias.

Diapositiva 17 – ReservaTest

ReservaTest comprueba el funcionamiento de las reservas.

Se prueba reservar un asiento, evitar duplicados, listar reservas de un usuario y cancelar reservas.

También se comprueba que no se puedan cancelar reservas de otros usuarios.

Diapositiva 18 – CompraTest

CompraTest valida el proceso de compra a partir de una reserva.

Comprueba que no se pueda comprar si la reserva ha caducado o no es del usuario.

También revisa que se genere el código QR al completar la compra.

Diapositiva 19 – LoginRoutesTest

LoginRoutesTest comprueba las rutas básicas de la web.
Verifica que la página principal tenga enlace al login y que la página de login cargue correctamente.

Diapositiva 20 – ExampleTest

ExampleTest es una prueba muy básica.
Solo comprueba que la página principal responde y funciona. Sirve para ver que el sistema está activo.

Diapositiva 21 – ReservaServiceTest

ReservaServiceTest comprueba la lógica de las reservas.
Se prueba reservar, evitar duplicados, cancelar reservas y listar las activas.
Aquí no interviene la parte web, solo la lógica interna.

Diapositiva 22 – CompraServiceTest

CompraServiceTest revisa el proceso de compra.
Se prueba que la compra funcione correctamente, que no se permita si la reserva ha caducado y que se gestione bien si algo falla.

Diapositiva 23 – LiberarReservasServiceTest

LiberarReservasServiceTest comprueba que las reservas expiradas se liberen bien.
También asegura que no se modifiquen reservas ya confirmadas o compradas.

Diapositiva 24 – IsAdminMiddlewareTest

IsAdminMiddlewareTest revisa el acceso a rutas de administrador.
Permite el acceso a admins y bloquea a usuarios normales con un error 403.

Diapositiva 25 – ModeloTest

ModeloTest comprueba que los modelos funcionan bien.

Se revisan cosas como borrado lógico, disponibilidad de asientos, tiempos de reserva y códigos QR.

Diapositiva 26 – SectorDimensionsTest

SectorDimensionsTest comprueba el cálculo de los asientos de un sector.

Se valida que filas, columnas y total de asientos se generen correctamente para el mapa del recinto.

AHORA QUE YA HEMOS TERMINADO DE LEER LOS MODELSO Y QUE LA PAGINA YA DEBERIA DE ESTAR EN MARCHA , vamos a ver el resultado de los tests!

[Diapositiva 27 – Página web: puesta en marcha]

vaya todos positivos que bien. entonces procederemos a analizar la pagina web!

[Diapositiva 28 – Home]

URL: <http://localhost>

Esta es la página principal o portada. Muestra los eventos destacados, información general del Roig Arena y enlaces a las secciones públicas. desde esta pagina el usuario tiene facil acceso a los eventos disponibles.

[Diapositiva 29 – Login]

URL: <http://localhost/login>

Formulario de autenticación. Credenciales de prueba:

- Administrador: [admin@admin.com](#) / [admin](#)
 - Usuario normal: [juan@example.com](#) / [password](#)
-

[Diapositiva 30 – Registro]

URL: <http://localhost/register>

Permite crear una nueva cuenta de usuario. El sistema validará que el email no exista previamente y que las contraseñas coincidan.

[Diapositiva 31 – Eventos (listado)]

URL: <http://localhost/eventos>

Muestra todos los eventos públicos disponibles ordenados por fecha. Desde aquí se accede al detalle de cada evento.

[Diapositiva 32 – Evento (detalle)]

URL: <http://localhost/eventos/2>

Pantalla con la información completa de un evento concreto: nombre, descripción, fecha, hora, ubicación. También incluye un botón para iniciar la compra de entradas.

[Diapositiva 33 – Compra (selección de asientos y pago)]

URL: <http://localhost/compra/2>

Aquí el usuario puede ver el mapa de asientos del evento, seleccionar sus localidades, confirmar la reserva y proceder al pago simulado.

[Diapositiva 34 – Dashboard (requiere login)]

URL: <http://localhost/dashboard>

Panel de control del usuario autenticado. Muestra un resumen de sus reservas activas, entradas compradas y pagos pendientes.

[Diapositiva 35 – Perfil (requiere login)]

URL: <http://localhost/profile>

Permite al usuario modificar sus datos personales, contraseña e información de contacto.

[Diapositiva 36 – Mis eventos (requiere login)]

URL: <http://localhost/mis-eventos>

Listado de eventos para los que el usuario ha comprado o reservado entradas.

[Diapositiva 37 – Mis eventos info (requiere login)]

URL: <http://localhost/mis-eventos-info>

Información agregada de todos los eventos en los que el usuario participa.

[Diapositiva 38 – Mi evento info (requiere login)]

URL: <http://localhost/mi-evento-info/{id}>

Detalle de un evento específico del usuario, incluyendo el código QR de su entrada, asiento asignado, etc.

[Diapositiva 39 – Mis pagos pendientes (requiere login)]

URL: <http://localhost/mis-pagos-pendientes>

Lista de reservas que aún no han sido pagadas y que están dentro del plazo de expiración. Aquí el usuario puede completar la compra.

[Diapositiva 40 – Acciones de usuario (no son páginas, son peticiones API)]

A continuación se describen las acciones que el usuario puede realizar mediante peticiones HTTP (no son páginas independientes):

- **POST** <http://localhost/logout> – Cierra la sesión del usuario actual.
 - **DELETE** <http://localhost/entradas/{id}> – Permite cancelar o eliminar una entrada (compra confirmada) del usuario, siempre que el evento no haya pasado.
 - **POST** <http://localhost/mis-pagos-pendientes/pagar> – Confirma el pago de una reserva pendiente, convirtiéndola en entrada definitiva.
-

[Diapositiva 41 – Administración (requiere login + rol admin)]

Todas las rutas que comienzan con **/admin** requieren que el usuario esté autenticado y que tenga el rol de administrador.

A continuación se detallan las funcionalidades exclusivas para gestores del recinto.

[Diapositiva 42 – Crear evento (página + acción)]

Página: <http://localhost/admin/eventos/create> – Formulario para introducir los datos de un nuevo evento.

Acción: **POST** <http://localhost/admin/eventos> – Procesa el formulario y almacena el evento en la base de datos.

[Diapositiva 43 – Actualizar evento (acción)]

Ruta de actualización: **PATCH** <http://localhost/admin/eventos/{id}>

Permite modificar todos los campos de un evento existente: título, fecha, descripción, aforo, etc.

(Nota: en la diapositiva aparece una URL de ejemplo de imagen, pero no forma parte del contenido académico.)

[Diapositiva 44 – Borrar evento (acción)]

Acción: **DELETE** <http://localhost/admin/eventos/{id}>

Elimina un evento del sistema. Se aplica un borrado lógico (soft delete) para mantener la trazabilidad, pero el evento deja de ser visible para los usuarios.

[Diapositiva 45 – Editor visual de sectores (página)]

URL: <http://localhost/admin/eventos/{eventoId}/sectores/editor>

Esta página permite al administrador gestionar la disposición de los asientos. Desde un editor visual se pueden añadir, eliminar o modificar sectores, así como definir rangos de filas y columnas. La interfaz facilita la configuración del recinto sin necesidad de tocar la base de datos directamente.
